Agentes de IA no SOC. Isso funciona?

Uma análise de um agente de IA na triagem do SOC

Rômulo Rocha
github.com/romrocha
linkedin.com/in/romrocha

YSTS 2026 !!! =)

POR QUÊ ESSA PESQUISA?

01 **SOC stress**
Alertas duplicados, semelhantes, falso-positivos, volume crescent de alertas, >80% dos alertas falso positivos, escalar somente com pessoas tem sido difícil, exaustivo e levado a níveis altos de turnover.

02 **Falta de medição pública**
“AI-powered SOC” carece de números abertos sobre eficiência, deixando a decisão restrita a provas de conceito “fechadas”, que por vezes podem ser custosas e com limitação de tempo, técnica ou dados suficientes.

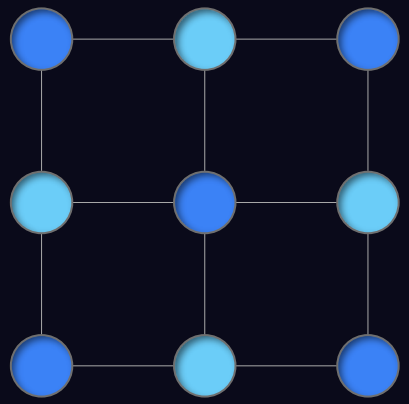
03 **Crescimento de maturidade da arquitetura agêntica**
Frameworks de desenvolvimento de agentes ganharam maturidade, com ferramentas para observabilidade, customização e farta documentação pública.

04 **LLM e RAG trazem capacidades novas**
Embeddings para correlação semântica, raciocínio sobre evidências, janelas de context maiores , etc

05 **Reasoning models vs standard models**
Em Agosto/2025, surgiram os modelos de raciocínio, seriam melhores para tarefas como triagem no SOC?

BLOCO 01

01



ARQUITETURA DO AGENTE

Como funciona, o que é código e o que é LLM, desafios de engenharia.

COMO FUNCIONA UM AGENTE

PROMPT + MEMÓRIA

System prompt
+ histórico de mensagens
+ resultados anteriores *(Memória)*
+ tool definitions

→

LLM (OpenAI)

Recebe contexto completo e decide sobre o próximo passo

Decide: tool_call ou resposta final

→

Tool_call

LLM escolhe qual função chamar e com quais argumentos

search_similar_alerts(query="lateral movement")

→

FERRAMENTA

Executa a ação e retorna resultado estruturado (JSON)

ChromaDB, SQLite, API externa...

↑ ↓

← resultado volta como contexto para o próximo turno (loop ReAct) ←

Agente = LLM + memória + tools + loop de raciocínio.
Uma pilha enorme de contexto, fluxo de entradas e saídas, para que o LLM busque a resposta mais próxima dentro do seu vetor de conhecimento.

O QUE É CÓDIGO vs O QUE É LLM

Nem tudo no agente depende do modelo — boa parte da confiabilidade vem de controle determinístico.

DETERMINÍSTICO (CÓDIGO)

- **Grafo de estados (LangGraph)**
  Ordem dos nós, transições, condições de saída
- **Guardrails e limites**
  max_loops, recursion_limit, retry guard, clamp
- **Tool routing e execução**
  ToolNode dispara, valida schema, retorna resultado
- **Vector search e ingestão**
  ChromaDB query, chunking, embedding
- **Persistência e estado**
  SQLite thread-local, checkpointing, schema do state
- **Output parsing e validação**
  Structured output, clamp de confidence, sanity checks

NÃO-DETERMINÍSTICO (LLM)

- **Triagem e classificação do alerta**
  Decidir TP/FP, severidade, confiança
- **Correlação semântica**
  Agrupar alertas por narrativa, não somente por IOC
- **Decisão de qual tool chamar**
  Tool_call e escolha do modelo via ReAct
- **Raciocínio e justificativa**
  "Por quê esse alerta é True Positive?" — texto livre gerado
- **Interpretação de contexto (RAG)**
  O que fazer com os chunks retornados do vector store?
- **Atribuição de MITRE ATT&CK**
  Mapear comportamento observado para tecnica/tática

INSIGHT: Código controla o "como executa". LLM controla o "o que fazer com a informação". **A confiabilidade do agente vem da qualidade das duas camadas.**

STACK, GRAFOS & NÓS

LangGraph	ChromaDB	SQLite	LangSmith	OpenAI
Orquestração do agente	Vector store + MiniLM-L6-v2	Estado persistido	Observabilidade	gpt-4.1-mini / gpt-5-mini

Grafos

OUTER GRAPH (ROUND)

`init` → `select_seeds` → `investigate` → `post_iteration` → `loop | finalize`

INNER GRAPH (UMA INVESTIGAÇÃO)

`investigate` → `correlate` → `decide`

↺ edge condicional (max_loops=3)

Cada nó = create_react_agent (padrão canônico LangGraph)

Nós do Analista de Triagem

select_seeds	=	Scan da fila + priorizacao
investigate	=	Abrir alerta + pivotar entidades
correlate	=	Validar shared observables, temporal proximity
decide	=	Registrar campanha / FP / merge

Mapeamento explícito ao fluxo humano

02

A PESQUISA

Dataset, calibração pré-experimento, vetor temporal.

DATASET — DESENHO FIXO, TEMPO COMO ÚNICA VARIÁVEL

FILOSOFIA DE DESIGN

SINTÉTICO (JSON)

Reproducibilidade total, ground truth controlado, zero PII, isolamento de variáveis entre rounds. **O custo é abrir mão de realismo adversarial.**

THE HIVE SCHEMA

Plataforma open-source amplamente usada em SOCs corporativos. TLP, Flag, Status, SourceRef, Observable types (url, domain, ip, hostname, file, etc)

100 ALERTS / ROUND

70 ruído (FP plausiveis) + 15 ransomware finance + 15 insider threat HR. Múltiplas fontes (EDR, IdP, Email GW, DLP, proxy, scanner). MITRE com overlap proposital.

REALISMO E VERIFICAÇÃO DE ESTRUTURA

Uso de repositórios públicos para trazer realismo de conteúdo para os alertas (Sigma Rules, Microsoft Sentinel Repo, Defender etc). **12 Vendors, 47 técnicas MITRE, 215 Observáveis, 76 IOCs e 11 fontes públicas de validação.**

IOC ENRICHMENT FRACO POR ESCOLHA

Só hostname e user repetem entre alertas de uma campanha. IPs, dominios, URLs, files, hashes: ocorrência única. Agente é forçado a correlacionar por (host, user, narrativa, semantica) — dificultando o cenário.

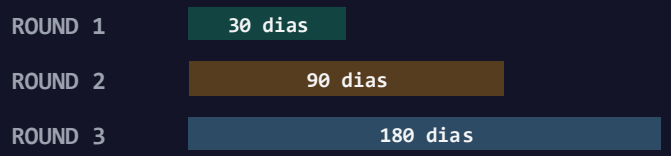
RUÍDO DIFÍCIL + CROSS-CONTAMINATION

Sem linguagem "auto-declarada" de baixa severidade. **Entidades sobrepostas entre ruído e campanha impedem classificação trivial por isolamento.**

CAMPANHA MALICIOSA

- Ransomware LockBit Style - phishing no usuário b.smith (T1566), roubo de credenciais (T1003), movimento lateral via SMB (T1021) e ransomware no file server (T1486).
- HR Sensible Information Extraction - insider exfiltrando dados de RH — h.potter em HR-LT-118 baixa compensation fora do horário (T1078, T1530) e exfiltra via Dropbox e USB (T1567, T1052).

VARIÁVEL EXPERIMENTAL



mesma composição, campanhas, regras de enrichment

HIPÓTESE

Janela temporal cresce → âncora temporal se dilui → agente perde dimensão de correlação e precisa se apoiar mais em sinais semânticos e observáveis. Se conseguir resistir, existe uma correlação semântica de verdade.

Detalhes, ground truth e regras de geração: github.com/romrocha/research_dataset_2026 | Limitacoes e detalhes no repo.

PRÉ-EXPERIMENTO E EXPERIMENTO

PRÉ-EXPERIMENTO – AGENT FINE TUNING

- +10 rounds de calibração na fase final pré medição comparativa.
- **Decisões arquiteturais movem métricas significativamente. (+9 pp accuracy, -66% missed threats)**
- **Análise profunda dos traces no LangSmith, acoplado a um agente de IA.**
- Tipagem de Tools com Pydantic. Schemas validados e retorno útil das ferramentas estabilizam o caminho cognitivo do agente.
- Pre-Load de decisões prévias em prompt agilizam

~ $15 Doláres de custo

O EXPERIMENTO!

- 600 alertas avaliados, em turnos de 200. Mesmo dataset canônico, mesmas tools e arquitetura. Só varia o LLM (4.1 e 5 mini) e o vetor temporal entre os rounds.
- Instrumentação completa via LangSmith. Auditoria por run de tokens, iterações, tool calls, latência por nós , custo, etc.
- Pipeline reproduzível end-to-end. *(Geração do dataset -> notebook de execução -> emissão de relatórios -> ground truth comparativo.)*
- Geração de métricas: acurácia, F1 (campaign detection e noise filtering), missed threats, latência, custo e CWTS.

~ $6 Doláres de custo

INSIGHT: ~3x mais investimento em calibrar do que em medir. Em SOC reais, a calibração e medição deve ser constante, e os custos também.

03

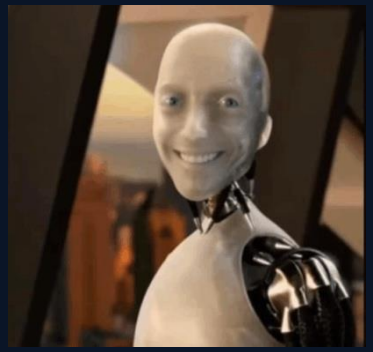
RESULTADOS

Matriz, custo, latência e dados observados.

MISSÃO PRIMÁRIA · DETECÇÃO

Sim, funciona.

Ambos os modelos detectaram as 2 campanhas reais em todos os 3 rounds.



CAMPANHA A — Ransomware financeiro

b.smith · FS-FIN-01 · estilo LockBit

Initial Access	Credential Access	Lateral Movement	Impact
T1566	T1003	T1021	T1486

CAMPANHA B — Insider exfiltration

h.potter · HR-LT-118 · Dropbox + USB

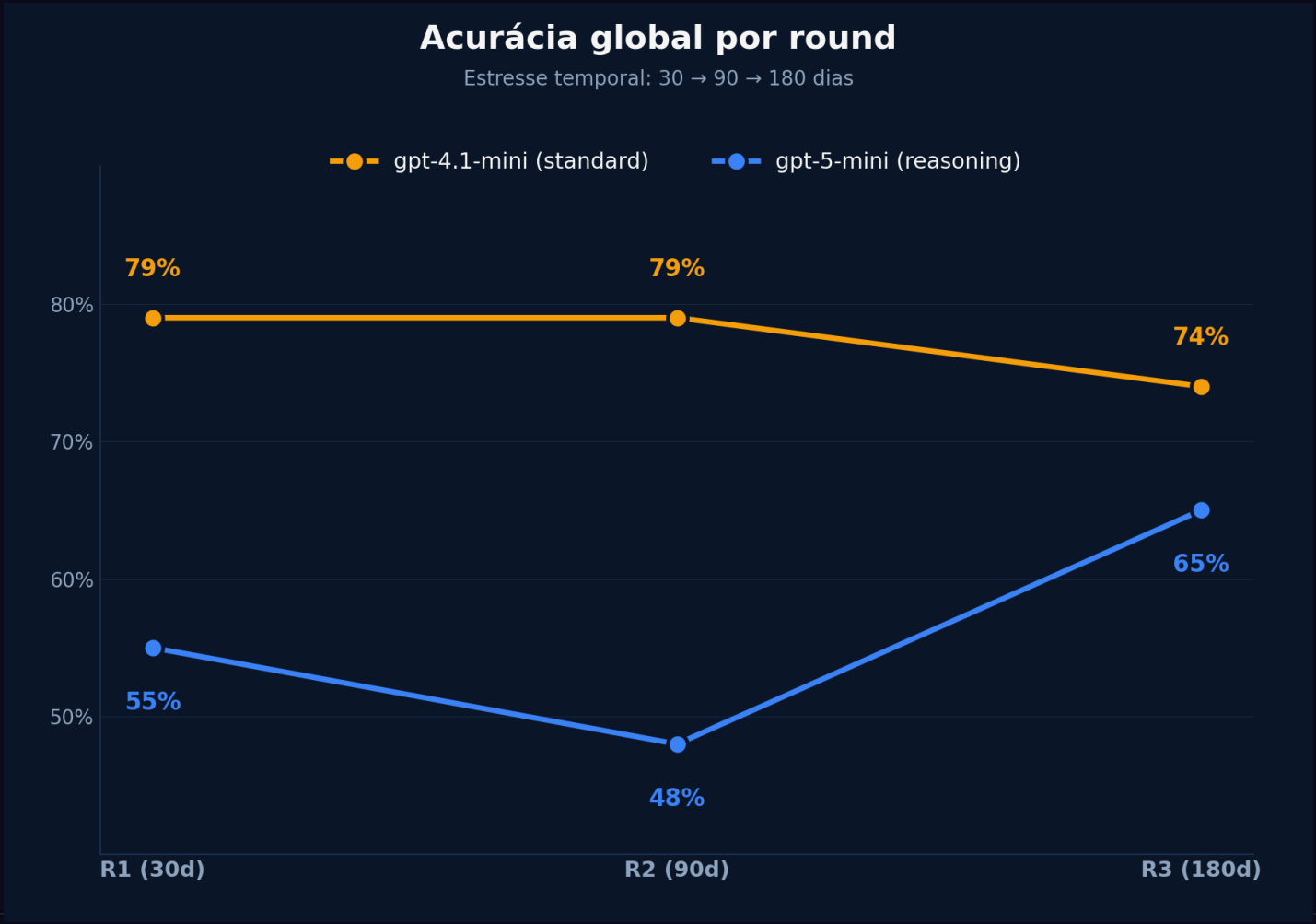
Initial Access	Collection	Staging	Exfiltration
T1078	T1530	T1560	T1567

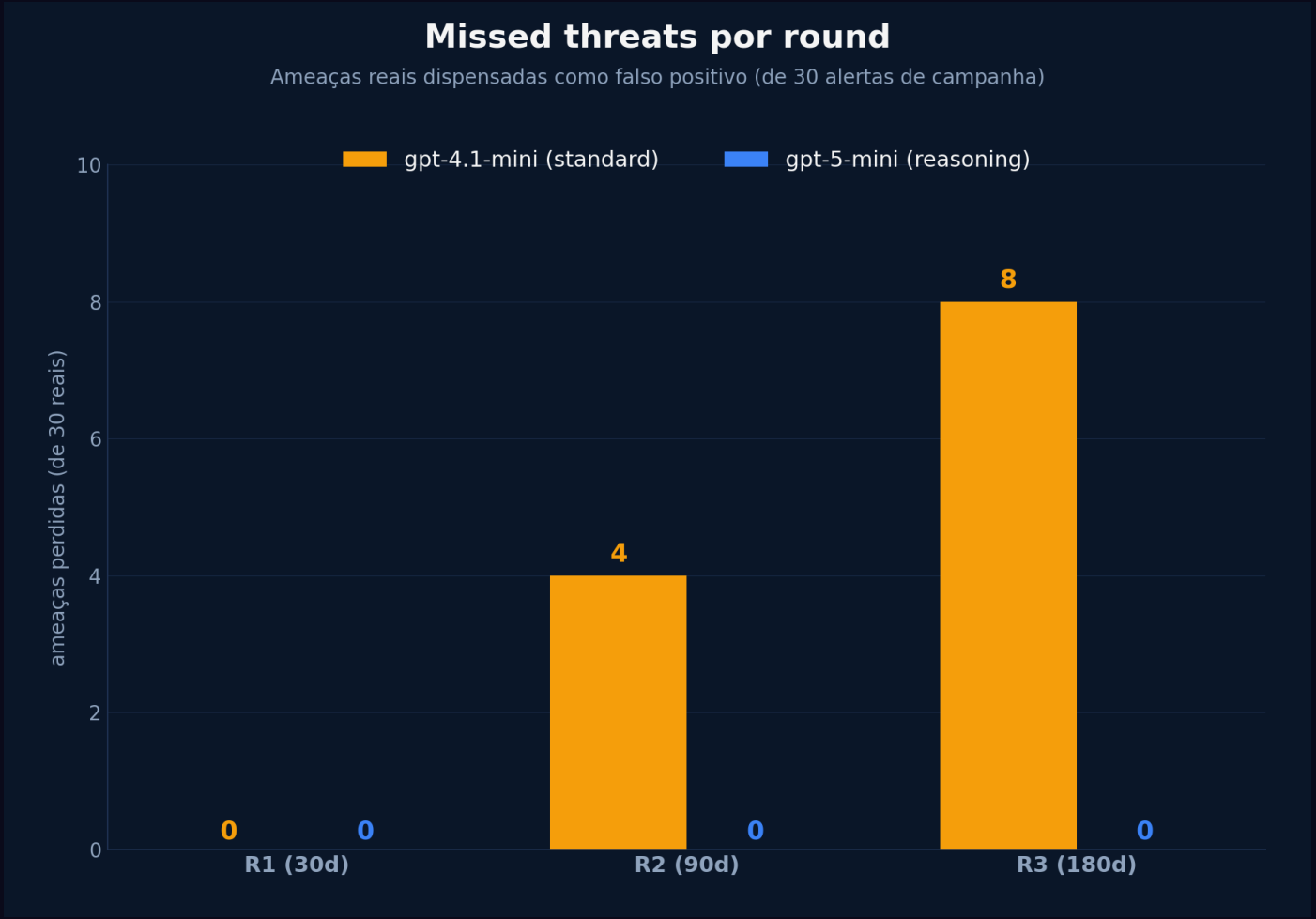
	R1 (30 dias)	R2 (90 dias)	R3 (180 dias)
CAMPANHA A	✓ DETECTADO	✓ DETECTADO	✓ DETECTADO
4.1-mini	Recall: 15/15	Recall: 13/15	Recall: 11/15
	Pureza: cluster limpo	Pureza: +2 extras	Pureza: +4 extras
5-mini	Recall: 15/15	Recall: 15/15	Recall: 15/15
	Pureza: +8 extras	Pureza: +10 extras	Pureza: +10 extras
CAMPANHA B	✓ DETECTADO	✓ DETECTADO	✓ DETECTADO
4.1-mini	Recall: 15/15	Recall: 13/15	Recall: 11/15
	Pureza: cluster limpo	Pureza: +1 extras	Pureza: +4 extras
5-mini	Recall: 15/15	Recall: 15/15	Recall: 15/15
	Pureza: +8 extras	Pureza: +12 extras	Pureza: +0 extras

Em SOC real, detectar é o ponto de partida — o L2 investiga e contém.

A pergunta deixa de ser 'funciona?' e passa a ser 'qual trade-off você aceita?'

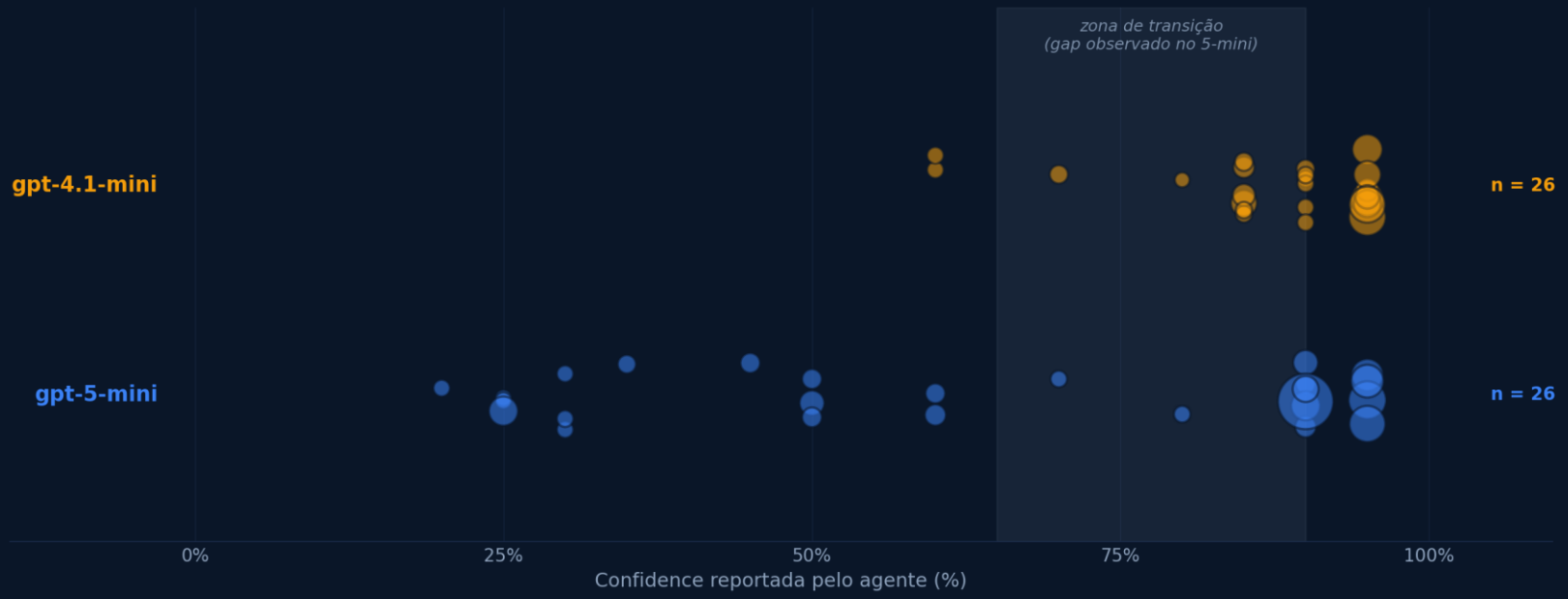
Recall = capturados/15 reais · Pureza = qualidade do cluster


Acurácia global por round
Estresse temporal: 30 → 90 → 180 dias
gpt-4.1-mini (standard)
gpt-5-mini (reasoning)
80%
70%
60%
50%
79%
79%
74%
55%
48%
65%
R1 (30d)
R2 (90d)
R3 (180d)


Missed threats por round
Ameaças reais dispensadas como falso positivo (de 30 alertas de campanha)
gpt-4.1-mini (standard)
gpt-5-mini (reasoning)
ameaças perdidas (de 30 reais)
10
8
6
4
2
0
0
0
4
0
8
0
R1 (30d)
R2 (90d)
R3 (180d)

Confidence reportada por campanha — flat vs bimodal

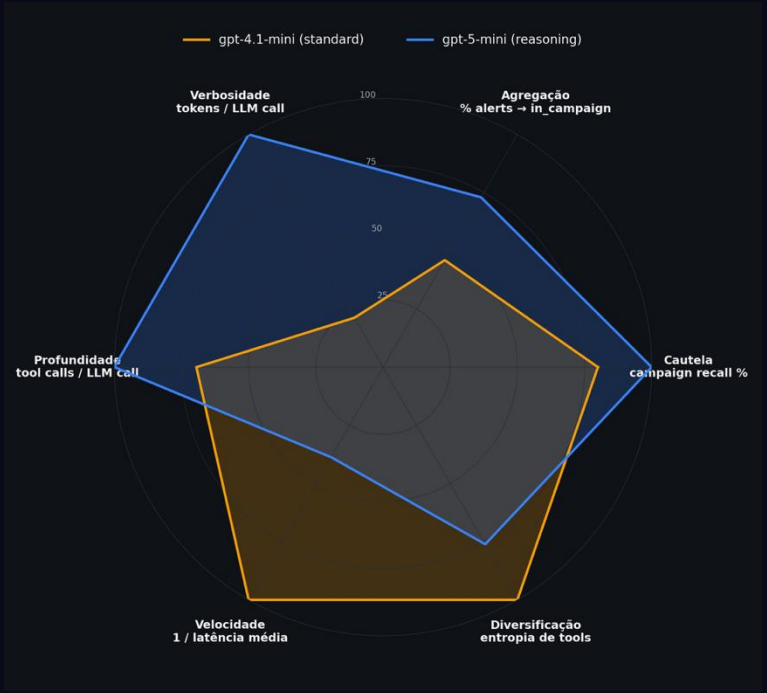
Cada ponto = 1 campanha detectada · tamanho = nº de alertas no cluster



4.1-mini reporta 85–95% para quase tudo (sem sinal útil). 5-mini é bimodal — gap observado em 65–90% separa os dois clusters. Threshold operacional exige calibração em ambiente real.

Perfil “cognitivo”durante round 3 (180d)

6 dimensões derivadas dos traces LangSmith e métricas de avaliação · escala 0–100 normalizada



7 DIMENSÕES METACOGNITIVAS

	gpt-4.1-mini	gpt-5-mini
Investigação	breadth-first	depth-first
Agregação	40/100	65/100
Velocidade	rápido (3,64s)	deep (7,86s)
Profundidade (tool calls)	1,02x	1,39x
Diversificação (tools)	H=3,26	H=2,68
Postura	balanceado	defensivo
Cautela	73,3%	100%

Os dois agentes simulam dois perfis de N1 reconhecíveis em equipes humanas: o produtivo (4.1) e o cauteloso (5-mini).

RESULTADOS

Custo e latência

6 runs (3 rounds × 2 modelos). Reasoning custa mais por iteração — mas o total da corrida não dispara.

CUSTO POR RUN (USD)

média sobre 3 rounds · range = min → max

gpt-4.1-mini

média
$0.81

RANGE
$0.76 → $0.85

P50
$0.75

gpt-5-mini

média
$1.17

RANGE
$1.10 → $1.25

P50
$0.80

LATÊNCIA POR ITERAÇÃO (s)

tempo médio do agente decidindo 1 alert · range = min → max

gpt-4.1-mini

média
2.8s

RANGE
0,4s - 45,7s

P50 · P95
1.6s - 8.7s

gpt-5-mini

média
8.2s

RANGE
0,8s - 66,7s

P50 · P95
5s · 26s

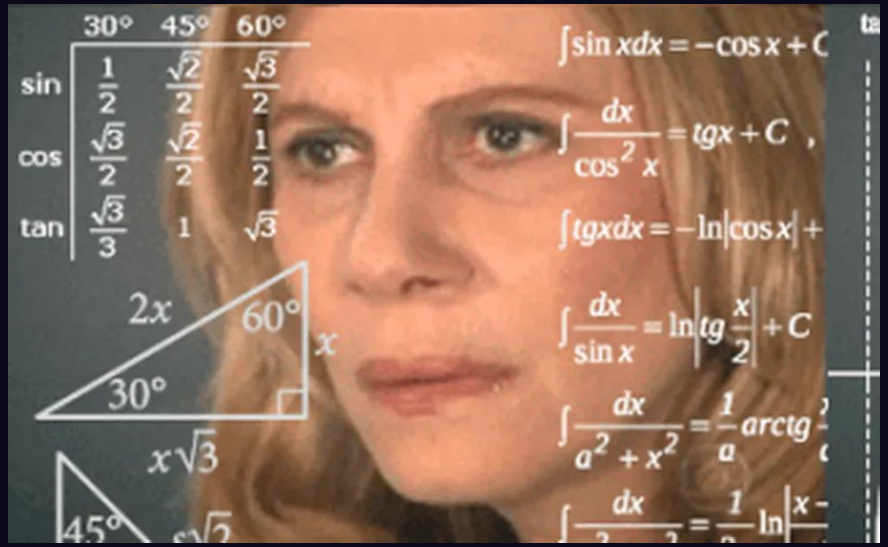
Cache hit ~56% · reasoning tokens (5-mini) explicam 60% mais latência por iteração mas total similar por run.

INSIGHT: ***Diferença de +45% no custo entre gpt4.1 e 5-mini, e de +69% na latência. (total de 4h27mins) de compute time.***

Oxi, o gpt4.1 é melhor?

04

CWTS



COST WEIGHTED TRIAGE SCORE. Quanto dói um falso negativo?

A acurácia esconde dois problemas estruturais

01

DESBALANCEAMENTO DE CLASSES

Acurácia recompensa o classificador preguiçoso.

Em SOC real, ruído domina. Um agente que sempre prevê "false_positive" acerta ~99% — sem fazer trabalho cognitivo. Bayes mostra que PPV depende da base rate, não da TPR.

Axelsson (2000) — The Base-Rate Fallacy and the Difficulty of Intrusion Detection Systems

02

ASSIMETRIA DE CUSTO

FN e FP não custam a mesma coisa. Acurácia trata como 1:1.

FP custa horas de analista. FN deixa ameaça passar — dwell time, incidente consumado. Literatura sugere razões de 10:1 a 100:1.

Elkan (2001) · Stolfo et al. (2000) · Gaffney & Ulvila (2001)

▼ **CWTS endereça os dois — normalização cobre (1), peso k cobre (2)**

Calculando a assimetria

CWTS – custo assimétrico vira score

Exemplificação do cálculo do custo assimétrico do SOC.

A INTUIÇÃO:

Cada FP custa **1 ponto**.

Cada FN custa **50 pontos**.

Vence quem tem **menor custo total**.

"O k=50 não é meu — é o midpoint da faixa documentada por Gaffney & Ulvila há 24 anos. Eles trouxeram teoria da decisão para IDS em 2001. Estou aplicando o framework deles em agentes de IA — não inventando nada, só estendendo."

EXEMPLO APLICADO AO ROUND 1:

gpt-4.1-mini = 18FP × 1 + 8FN × 50 **= 418**

gpt-5-mini = 35FP × 1 + 0FN × 50 **= 35**

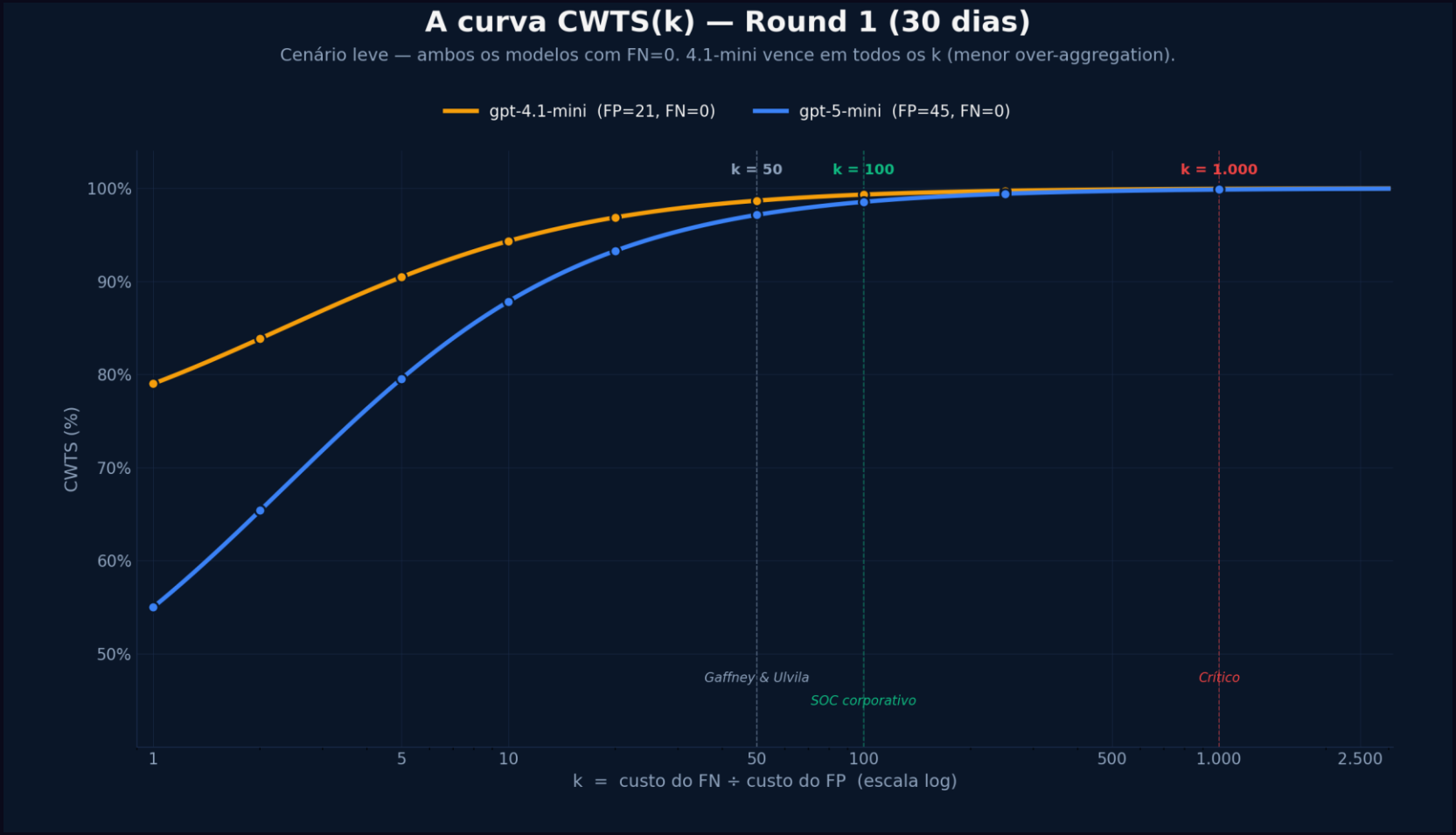
Pior caso = 70 × 1 + 30 × 50 = 1570

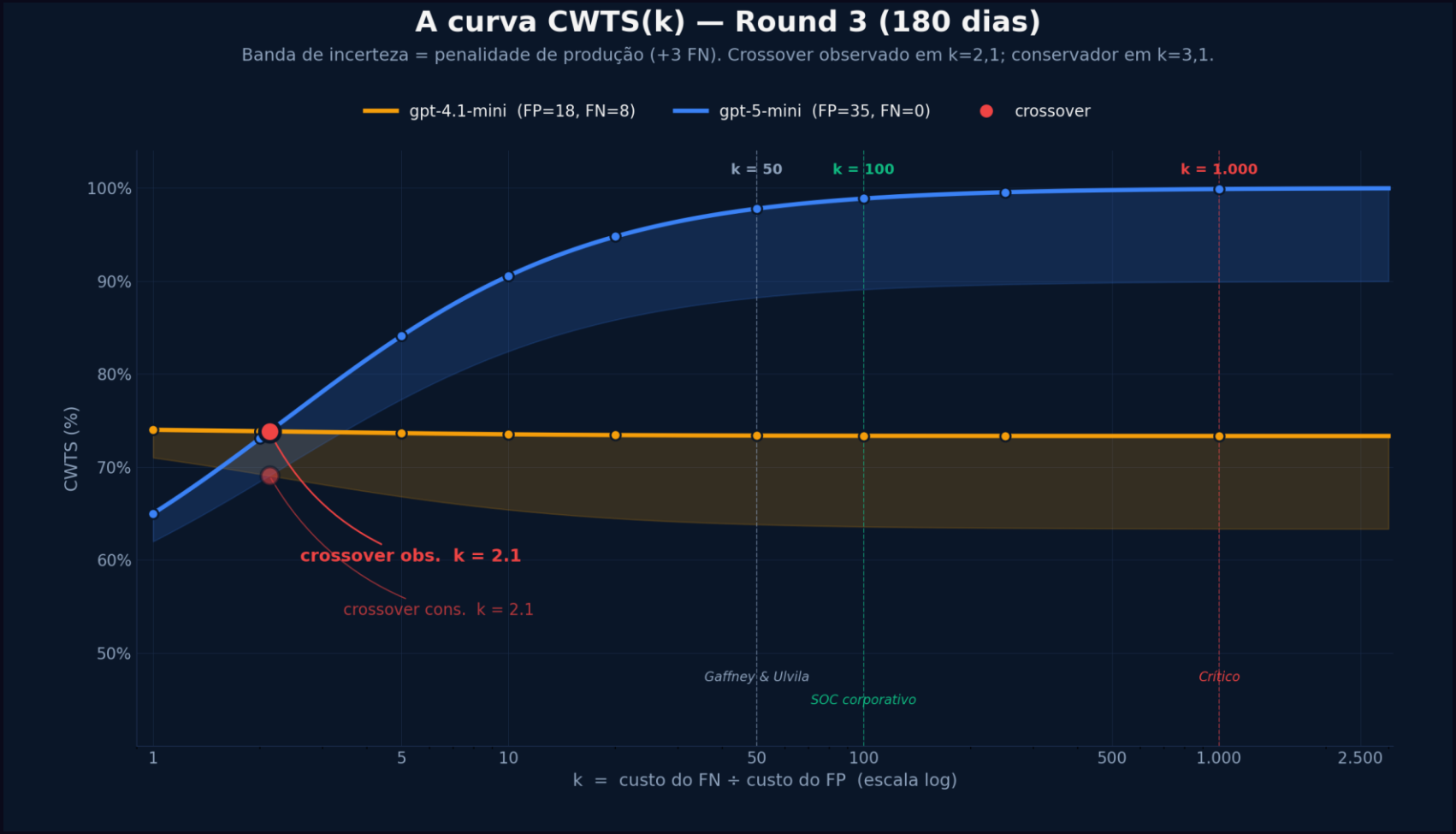
CWTS = 1 − custo / pior_caso

4.1-mini → **73,4%**

5-mini → **97,8%**

Elkan (2001) · Stolfo, Fan, Lee et al. (2000) · Gaffney & Ulvila (2001)


A curva CWTS(k) — Round 1 (30 dias)
Cenário leve — ambos os modelos com FN=0. 4.1-mini vence em todos os k (menor over-aggregation).
gpt-4.1-mini (FP=21, FN=0)
gpt-5-mini (FP=45, FN=0)
k = 50
k = 100
k = 1.000
100%
90%
80%
70%
60%
50%
CWTS (%)
Gaffney & Ulvila
SOC corporativo
Crítico
1
5
10
50
100
500
1.000
2.500
k = custo do FN ÷ custo do FP (escala log)


A curva CWTS(k) — Round 3 (180 dias)
Banda de incerteza = penalidade de produção (+3 FN). Crossover observado em k=2,1; conservador em k=3,1.
gpt-4.1-mini (FP=18, FN=8)
gpt-5-mini (FP=35, FN=0)
crossover
k = 50
k = 100
k = 1.000
100%
90%
80%
70%
60%
50%
CWTS (%)
crossover obs. k = 2.1
crossover cons. k = 2.1
Gaffney & Ulvila
SOC corporativo
Crítico
1
5
10
50
100
500
1.000
2.500
k = custo do FN ÷ custo do FP (escala log)

05

Fechamento

Insights e uso prático de agentes na triagem

FECHAMENTO — TL;DR

5 aprendizados da pesquisa

01 TRIAGEM AUMENTADA FUNCIONA

Agente de nível construído com LangGraph + tools tipadas + observabilidade entrega resultado mensurável. 6 runs, 600 alerts com acurácia superior a >70% sustentada nos 3 rounds , além de CWTS maior que 90% para K=50. Todas as campanhas maliciosas foram detectadas.

02 MODELOS DIFERENTES, NÃO HÁ UM VENCEDOR

Trocar de gpt-4.1-mini para gpt-5-mini, mudou o estilo cognitivo. Modelos de raciocínio funcionam fundamentalmente diferente, escolha de modelo é escolha de viés operacional.

03 REASONING MODEL REPORTA CONFIANÇA CALIBRADA, STANDARD NÃO

O gpt5-mini distribuiu a confiança em dois clusters bimodais, ele sabe quando duvidar da informação que produz. O 4.1-mini reporta uma confiança flat (85%-95%) o que pode ser um desafio para o nível 2 em priorizar.

04 CONFIABILIDADE DO AGENTE VEM DA ENGENHARIA, NÃO DO MODELO

Pre-tuning de tools tipadas (Pydantic), pré-load de decisões em prompt, observabilidade rigorosa (LangSmith), ground truth determinístico. +9 pp de accuracy vieram só dessa fase, antes de qualquer benchmark. Custo: $15. Engineering > prompt engineering.

05 MÉTRICAS PRECISAM RESPEITAR A ASSIMETRIA DO SOC

Acurácia trata FP e FN como iguais. SOC real não — perder uma ameaça custa 50× mais que investigar ruído. CWTS dá vocabulário matemático para essa assimetria.

FECHAMENTO — RISCOS

Antes de colocar IA no seu SOC

VARIÂNCIA PRODUTIVA

Qualidade do agente depende de fine-tuning, embeddings, prompts e dados reais. Benchmark sintético é piso teórico, não garantia.

Lembre-se
GIGO – Garbage in, Garbage out.

Métricas vistas aqui (77% / 0 missed) podem cair 10-30 pp em SOC real sem calibração específica.

COMPLEXIDADE DA PIPELINE

A complexidade do seu SOC vai aumentar. Quem vai revisar o output do agente? Como será feito esse finetuning? Que tipo de guardrails vai definir? Como medir que melhorou ou piorou?

É fundamental ter engenharia de IA ativa no time.

Custo de revisão precisa entrar no ROI. Pipeline cresce: observabilidade + auditoria + retraining contínuo.

VIÉS DE CONFIRMAÇÃO

Vimos que modelos standard reportam alta confiança em quase todos os resultados, com uma história coerente, analistas podem cair no viés de confirmação. O uso de modelos que imprimem confiança distribuída pode ajudar nessa análise mais cautelosa.

Treinar humanos a DUVIDAR do agente é parte do projeto. UX precisa mostrar incertezas, não esconder.

INSIGHT: *Não é por o agente acertar muito que ele deveria operar sozinho. Aonde ele erra são exatamente o que humano cobre.*

FECHAMENTO — APLICAÇÃO

Augmented, not autonomous. (yet?)

✓ PRONTO PARA USO HOJE

Baixo risco se errar. Humano valida output natural do workflow.

- Documentação automática de alertas (descrição, contexto, MITRE mapping)
- Enriquecimento (IOC lookups, geo, ATT&CK tactic, threat intel)
- Correlação determinística (mesma hash, mesma URL, mesmo IP)
- Sumarização de cases longos para handover entre turnos

⚠ CAUTELA E SUPERVISÃO

Erros caros. Humano sênior valida cada decisão.

- Triagem L1 completa autônoma (action-reason desync, over-aggregation)
- Decisão final de FP / escalation sem revisão humana
- Resposta automática a incidentes (containment, isolation)
- Multi-agent autônomo sem observabilidade entre etapas

DISCIPLINA DE ENGENHARIA — NÃO-NEGOCIÁVEL

- Observabilidade ponta-a-ponta (LangSmith, traces, métricas)
- Versionamento de prompts e schemas
- Monitoria contínua (purity, recall, missed, latência, custo)
- Auditoria periódica de qualidade das decisões (hallucinations, sycophancy, drift)

`FINAL BOSS INSIGHT:` *Se você não pode auditar o agente, você não deve usar o agente. SOC é disciplina, não experimento.*

Obrigado.
Q&A

Rômulo Rocha – linkedin.com/in/romrocha
github.com/romrocha